

Open source hardware

During your design and experimentation, you might have come across development boards like Beagleboard, Hawkboard or Arduino. These are called open source hardware because they all have the physical artefact of the technology (the hardware design and the tools used to design) open. Openness in terms of hardware can have a whole range of meanings. The licence for open source hardware (OSH) is the same as that for open source software. But one of the key differences is that unlike OSS, OSH cannot be completely free of cost because you will always have to pay for the components, the board, etc while using open source hardware.

Many embedded design houses have started using open source hardware for their design and development activities. The OSH community is actively contributing towards this.

We initially used `avr-gcc` for compilation. But when the code size reached the limit of the chip we were using, we decided to increase the optimisation levels in `avr-gcc` itself. However, this led to unpredictable results. We did some research and found that `avr-gcc` gave the best results when working on the lowest optimisation levels. Many other people had reported issues with increase in optimisation. Therefore we were left with no option but to switch to a proprietary tool. We switched to IAR, which reduced the final code size but still gave predictable results."

Uncertainty—too many results, versions. Open source initiative is driven by the users. So if the users are inactive, there will be no support available. Walia shares, "Support on open source could be very challenging as you hope that problems that you face have a solution on the Internet somewhere. In the absence of the same, you are left in the lurch and then you don't have any other choice but to move to proprietary."

Community support is available if you are active in the community. And because there are so many varied results, it may take you some time to figure out which is best suited for your case.

The other problem is fragmentation from both ends. Hills adds, "There are many versions and many people working on them. Therefore control of the build and maintenance is very difficult."

Sometimes in-time support also delays your development activity. Sharing his experience, Jain says, "We faced

another issue with `avr-gcc` regarding generation of a particular type of debugging file for JTAG debugging. The JTAG debugger (AVR Studio) from Atmel used elf files for JTAG debugging. However, when we added some modules in our code, a bug in `avr-gcc` got activated, due to which it stopped generating the elf file. The issue that a particular declaration of a structure in one of our modules was causing this problem was identified almost six months later (by which time we had started using IAR)."

Sometimes, you may find differences or incapacities among libraries used for development. This may lead to difficulties in embedded development.

Lack of testing. Proprietary tool vendors carry out extensive testing of tools provided, be they proprietary or mature open source tools. They also provide updates to new features through back ports. Proprietary tools are optimised for the speed and size of specific processor or application. Certain popular open source tools for embedded projects are tested extensively by the users around the globe, while others may not be tested enough to give you an appropriate result.

Is open source fit for embedded?

It is a question that will have different answers depending on a number of factors. Every embedded project is handled differently and priorities in each case are also different. It is true that there are a large number of free (no money) open source tools.

Karthick feels, "One can get the best from the open source community only by being a part of it. There is a lot of action happening in the OS community, much more than is possible with having proprietary software. A lot of support is available, especially if you are active in the community. Of course, if you have downloaded free tools, you cannot really expect someone to spend his time supporting it, without a fee." But if you are smart enough to fix it, you can save on support as well.

Using open source may not always result in cost savings. Citing an example, Hills says, "There is one Asian car maker who has discovered that when they saved 40,000 Euro by buying cheap tools, it will cost them over 100,000 Euro per project to get the same results as the more expensive tools." According to their experience, they found that the gcc is 30 per cent less efficient than commercial embedded compilers. "Another problem is that you never know what you are getting. Depending on where you get them from, gcc ARM compilers will perform in the same way. Also, gcc-C is not ISO/ANSI-C. It may or may not work to the C standard."

For most design engineers, the experience of using open source for their embedded development was good, but surely not the best. Else, they would have used it without any problem for their production.

Vasa shares, "There is always a certain element of optimisation in speed and size essential to be carried out to ensure that these open source tools are used in production. Special groups dedicated to open source should be formed to upstream and downstream changes made to tools for maturity and feature upgradation, respectively. This is a good win-win strategy for corporate and open source community."

Note. For further discussions on the topic, join www.linkedin.com/groups?mostPopular=&gid=71510&goback=.nmp.*1.*1.gde.71510_member_26795944 ●